In the name of Allah

Retrieval Augmented Generation (RAG)

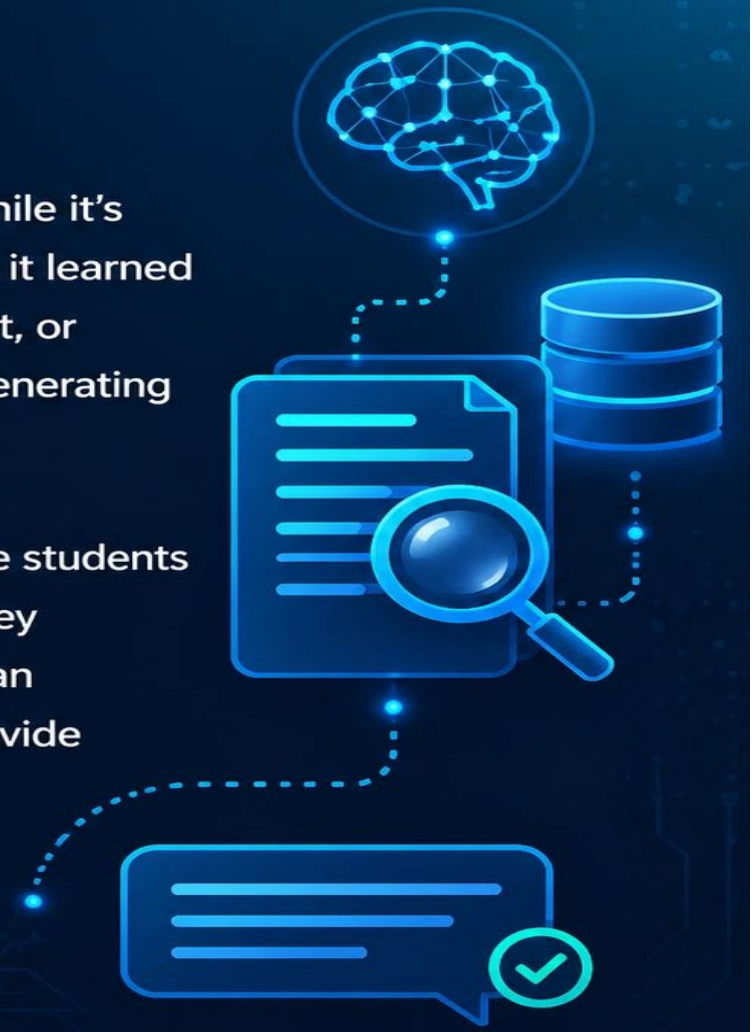
Lecturer: Mohammad Farhadi

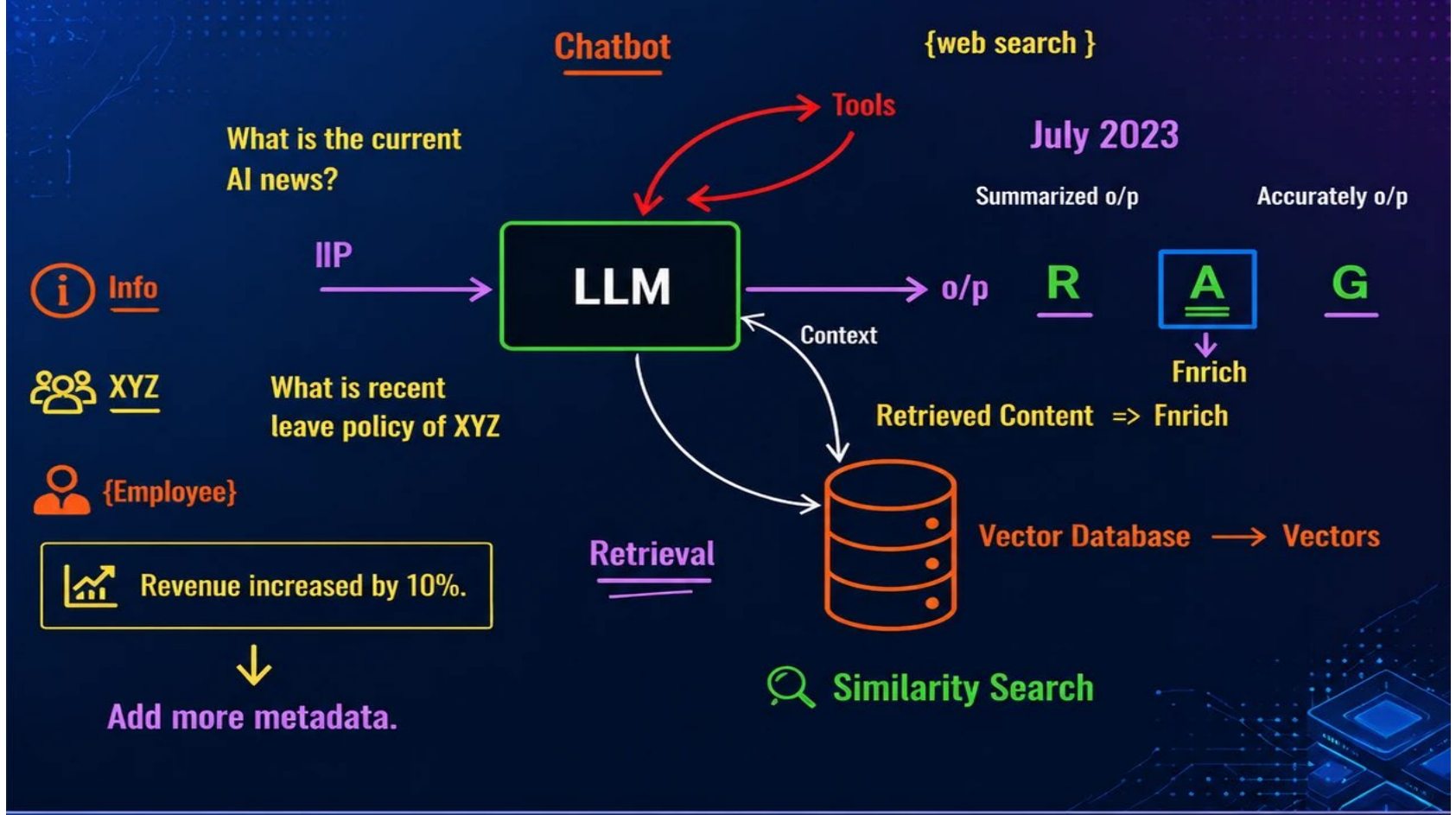
RAG (Retrieval-Augmented Generation) is a powerful technique that enhances AI language models by combining their generation capabilities with external knowledge retrieval.

What is RAG?

RAG is like giving an AI assistant access to a library while it's answering questions. Instead of relying solely on what it learned during training, the AI can now look up specific, current, or specialized information from external sources before generating its response.

Think of it this way: Traditional language models are like students taking a closed-book exam - they can only use what they memorized. RAG-enabled models are like students in an open-book exam - they can reference materials to provide more accurate, detailed, and up-to-date answers.





Augmented =

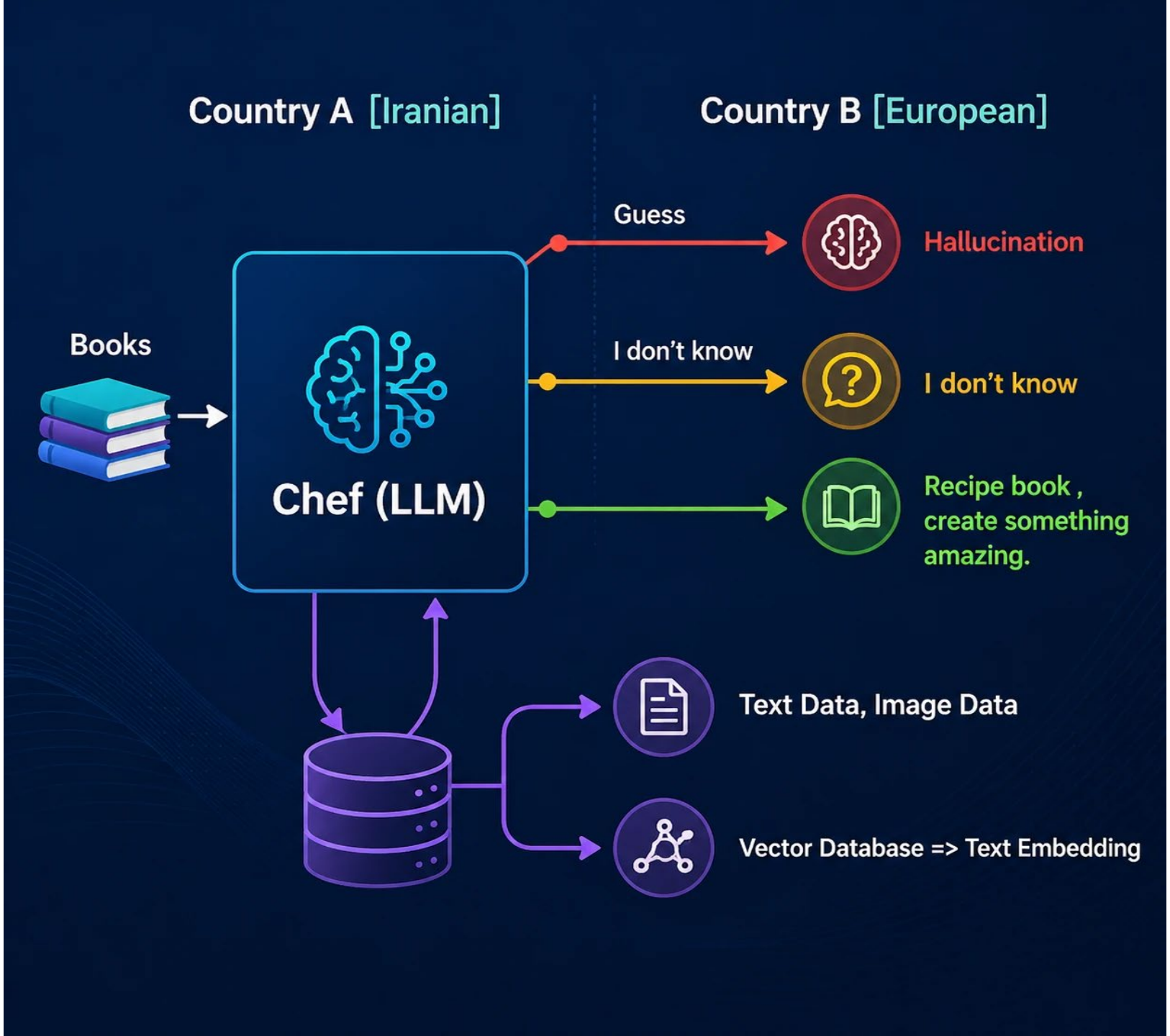
```
{
  "txt": "Revenue increased by 10%",
  "Source": "Tesla Annual Report 2023",
  "dat": ___
}
```

2) RAG

1) **Retrieval** – Finding Relevant Info **[Vector Databases]**

2) **Augmentation** – Enhancing Context with Metadata.

3) **Generation** – Producing the Answer.


Country A [Iranian]
Country B [European]
Guess
Hallucination
Books
I don't know
I don't know
Chef (LLM)
Recipe book , create something amazing.
Text Data, Image Data
Vector Database => Text Embedding

Customer Support

Without RAG

LLM ⇒ **Customer Support**

Customer: "What's your return policy for items bought during the Black Friday sale?"

AI: Generally, most companies offer 30-day returns, but policies may vary...
[Generic, unhelpful response]

With RAG

AI Assistant

AI Assistant
[LLM + Vector Database]

Customer: "What's your return policy for items bought during the Black Friday sale?"

AI: According to our current policy (Policy Doc v3.2, updated Nov 2024), Black Friday purchases have an extended 60-day return window until January 31st. Items must be unused with original tags. Electronics have a 15-day return period due to rapid depreciation. Would you like me to start a return for a specific item?"

Cost Savings: JPMorgan saved $150M annually by implementing RAG for research analysis instead of fine-tuning models monthly

Accuracy: Microsoft reported 94% reduction in AI hallucinations after implementing RAG in their Copilot products

Flexibility: Bloomberg updates their financial AI assistant hourly with new market data - impossible with traditional LLMs

Compliance: Healthcare companies use RAG to ensure AI responses always cite approved medical sources

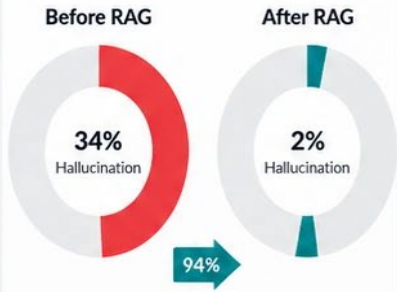
Why RAG Matters - The Business Impact

Real-world results from enterprise implementations

Cost Savings

JPMorgan Case Study:
Saved $150M annually using RAG for research analysis vs. monthly fine-tuning

Accuracy Improvement



Microsoft Case Study:
94% reduction in AI hallucinations after implementing RAG in Copilot

Update Flexibility

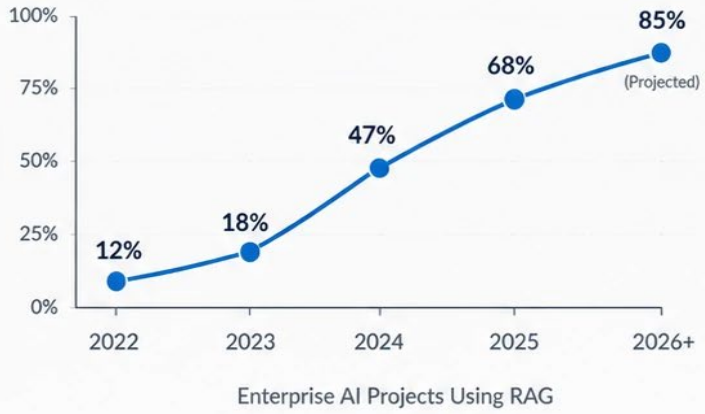
Bloomberg Case Study:
Updates financial AI assistant hourly with new market data via RAG

Compliance

- 100% Source Attribution
- Regulatory Compliance
- Complete Audit Trail
- Approved Sources Only

Healthcare Case Study:
RAG ensures all AI responses cite only FDA-approved medical sources

RAG Adoption Trends



Average ROI
312%
Within first year

Implementation Time
6-8 weeks
Average deployment

Top Industries Adopting RAG

- Finance (87%)
- Healthcare (76%)
- Legal (71%)
- E-commerce (65%)
- Manufacturing (58%)
- Education (52%)

Understanding Prompt Engineering vs Fine-tuning vs RAG

Prompt Engineering

"Teaching through instructions"

How it Works:

- Write specific instructions in your prompt
- Use examples (few-shot learning)
- Structure prompts with clear context
- Model remains unchanged

Pros:

- No technical expertise needed
- Instant results
- Free (no training costs)
- Highly flexible
- Works with any LLM

Cons:

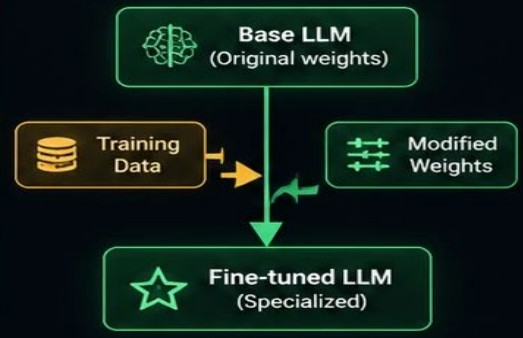
- Limited by model's base knowledge
- Inconsistent results
- Token limits restrict complexity
- Can't add new knowledge

Best For:

- Quick prototyping
- General-purpose tasks
- When you need flexibility
- Small-scale applications

Fine-tuning

"Teaching through training"



How it Works:

- Prepare domain-specific training data
- Train model on your data
- Model weights are permanently changed
- Creates a specialized version

Pros:

- Deeply specialized knowledge
- Consistent behavior
- No prompt engineering needed
- Can learn new writing styles
- Better for specific domains

Cons:

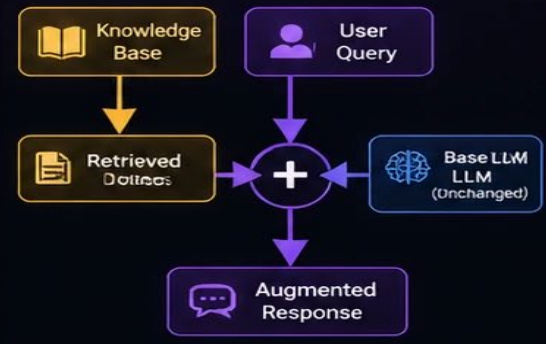
- Expensive ($1000s-$10000s)
- Requires ML expertise
- Can "forget" general knowledge
- Needs retraining for updates

Best For:

- Specific writing styles/tones
- Domain-specific language
- High-volume, consistent tasks
- When accuracy is critical

RAG

"Teaching through retrieval"



How it Works:

- Store documents in vector database
- Retrieve relevant docs for each query
- Combine docs with query as context
- LLM generates answer from context

Pros:

- Always up-to-date information
- No training required
- Source attribution/citations
- Can handle private/proprietary data
- Cost-effective

Cons:

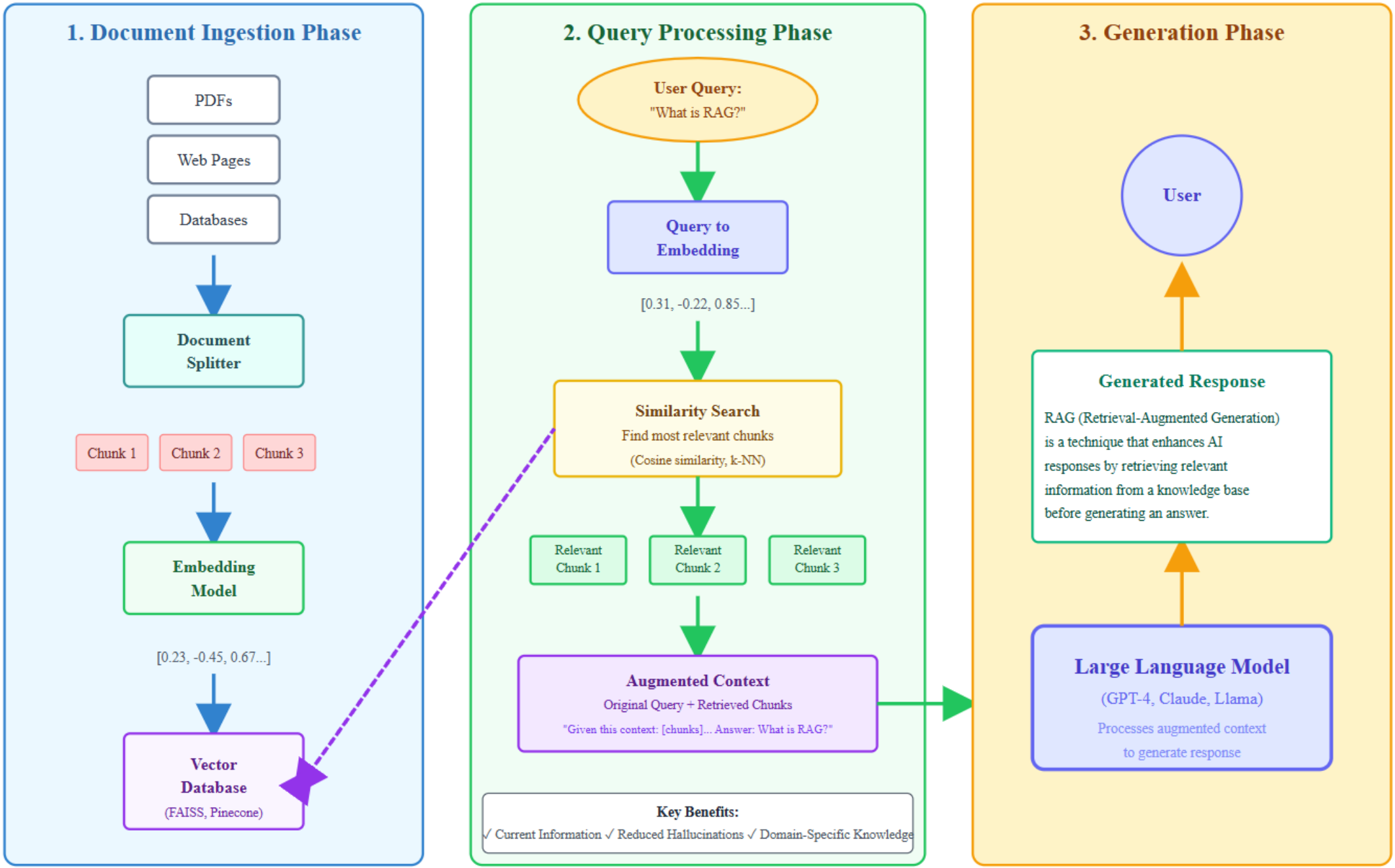
- Requires infrastructure setup
- Retrieval quality affects results
- Latency from retrieval step
- Context window limitations

Best For:

- Knowledge bases documentation
- Real-time/frequently updated info
- Customer support systems
- Compliance-heavy industries

Quick Decision Guide: Which Method Should You Use?

RAG (Retrieval-Augmented Generation) Architecture



LangChain Document Structure

Understanding the core components of LangChain Documents

LangChain Document

```
from langchain.schema import Document
```

Core Components:

- page_content (str)
- metadata (dict)

```python
# Creating a Document
doc = Document(
    page_content= "This is a technique...",
    metadata={
        "source" : "chapter1.pdf",
        "page" : 1,
        "timestamp" : "2024-01-15"
    }
)
```

page_content (String)

- The actual text content of the document
- Contains the main information to be embedded and searched
- Must be a string (can be any length)

Examples:

Research Paper:
"Natural-language-based Generative (NLG) combines the benefits of pre-trained language models with information retrieval and question in particular cases on specific and controlled responses..."

Product Manual:
"To install the software, first ensure your system meets the minimum requirements: Windows 10 or later, 8GB RAM, and at least 20GB of free disk space..."

Best Practices:

- ✓ Keep content focused and coherent
- ✓ Remove unnecessary formatting before storage
- ✓ Consider chunk size limits (typically 500–2000 tokens)

metadata (Dictionary)

- Additional information about the document
- Used for filtering, tracking, and context
- Can contain any JSON-serializable data

Common Metadata Fields:

source
File path or URL
"docs/chapter1.pdf"

page / chunk_id
Location in document
(page: 1, id: chunk_1)

timestamp
Creation or extraction date
"2024-01-15T10:30:00Z"

author
Document author
"John Doe"

category / type
Document classification
"technical", "legal"

language
Content language
"en", "es", "fr"

Tip: Add custom fields for your use case
Examples: department, security_level, version, keywords, embeddings_model

LangChain Document Loaders

PDFLoader

```
from langchain_community.document_loaders
import PyPDFLoader
loader = PyPDFLoader("file.pdf")
documents = loader.load()
```

CSVLoader

```
from langchain_community.document_loaders
import CSVLoader
loader = CSVLoader("data.csv")
documents = loader.load()
```

WebBaseLoader

```
from langchain_community.document_loaders
import WebBaseLoader
loader = WebBaseLoader("https://...")
documents = loader.load()
```

DirectoryLoader

```
from langchain_community.document_loaders
import DirectoryLoader
loader = DirectoryLoader("./docs")
documents = loader.load()
```

Additional Loaders:

- UnstructuredLoader (multiple formats)
- JSONLoader
- TextLoader
- GitbookLoader
- NotionDirectoryLoader
- GoogleDriveLoader
- AzureBlobLoader
- SlackDirectoryLoader
- S3FileLoader
- YouTubeLoader
- WikipediaLoader
- ArxivLoader
- ConfluenceLoader
- DatagmaiLoader
- EverNoteLoader
- HuggingFaceDatasetLoader

Document Transformers (Text Splitters)

CharacterTextSplitter

```
splitter = CharacterTextSplitter(
  chunk_size=200,
  chunk_overlap=20
)
```

RecursiveCharacterTextSplitter

```
splitter = RecursiveCharacterTextSplitter(
  chunk_size=200,
  chunk_overlap=20,
  separators=["\n\n", "\n", "", ""]
)
```

TokenTextSplitter

```
splitter = TokenTextSplitter(
  chunk_size=200,
  chunk_overlap=20,
  model_name="gpt-3.5-turbo"
)
```

SemanticChunker

```
splitter = SemanticChunker(
  embeddings,
  breakpoint_threshold_type="percentile"
)
```

```
# Split documents into chunks
chunks = splitter.split_documents(documents)

# Each chunk is a new document with preserved metadata
```

Embeddings

[cat, dog, kitten, puppy]

SQL

Cat ⟶ 

Yours : [cat]

Exact Matches.

[cat, dog, kitten, puppy]

Embeddings

Cat ⟶ 

Text ⟶ [ ]

Apple ⟶ [0.2, 0.3, 0.5]

↓

Vector representation

Find Similar Meanings

Yours = { Cat, Kitten }

Embedding Models ⟶ Huge Amount Data.

↓

Vector Representation.

OpenAI,

Huggingface

Cat , Kitten

[0.2, 0.3, 0.4]

[0.2, 0.4, 0.3]

⇒ Cosine Similarity

Feature Representation

Marvel	Action	Comedy	Suspense
A IronMan	0·95	0·2	0·6
B Hulk	0·96	0·4	0·7
C Sherlock Holmes	0·6	0·75	0·9

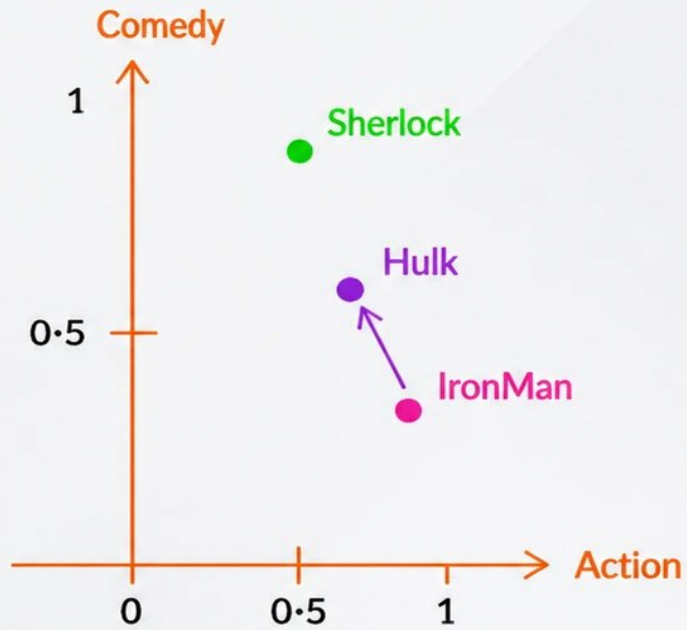
Netflix

IronMan

↓

Hulk

$$\text{Cosine Similarity} = \frac{A \cdot B}{||A||\ ||B||}$$



Embedding Model

OpenAI, Huggingface